

ANGULAR MASTER STUDY GUIDE

Comprehensive Examination Reference Manual

1. Introduction to Angular & Environment Setup

Overview of Angular

Angular is a powerful application-design framework for creating efficient and sophisticated single-page applications (SPAs) that can scale with both the size of a team and codebase.

Key features of Angular include:

- **Component-based architecture:** Easily split code into well-encapsulated, reusable parts.
- **TypeScript:** Built on TypeScript, which provides typing and a more concise syntax than plain JavaScript, leading to more maintainable code and fewer bugs.
- **Dependency Injection:** A system that makes it easy to share code and services across components, improving modularity and testability.
- **Routing:** A feature-rich navigation toolkit for managing application states and views.
- **Built-in security features:** Includes default security measures, such as HTML sanitization, to protect users from common vulnerabilities.

Setting Up the Development Environment — Installing Angular CLI

Before installing the Angular CLI, you must have Node.js installed, which includes the npm package manager.

- **Install Node.js and npm:** Download and install the active LTS (Long Term Support) version of Node.js from the official website.
- The npm command-line interface is installed with Node.js by default.
- Verify the installation by opening a terminal or command prompt and running the following commands:

```
node -v  
npm -v
```

Install the Angular CLI:

```
npm install -g @angular/cli
```

For Windows users, you might need to set the execution policy to allow PowerShell scripts to run:

```
powershell
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Verify the Angular CLI Installation:

Confirm the installation by running the `ng version` command in your terminal. This will display the installed versions of the Angular CLI, Node.js, and underlying Angular packages:

```
ng --version
npm ng --version
```

Creating a New Angular Project

```
ng new my-app
cd my-app
ng serve --open
```

Angular Project Structure and Files

Core Project Structure: A typical, scalable Angular 20 project structure looks like this:

- `/src` : Contains the application's source code. This is where most development work occurs.
- `/app` : The root application folder, structured into logical subdirectories.
 - `/core` : Houses singleton services and components that are instantiated once for the entire app (e.g., authentication services, interceptors, layout components).
 - `/shared` : Contains components, pipes, and directives that are used across multiple features and are truly reusable.
 - `/features` or `/pages` : Contains feature-specific code. Each feature (e.g., authentication, user management, products) has its own subdirectory, promoting lazy loading and clear boundaries.
 - `components` : for components specific to this feature.
 - `services` : for services specific to this feature.
 - `models` : for models/interfaces specific to this feature.
- `app.component.ts | html | scss | spec.ts` : The root component files for the application.
- `app.config.ts` : Used for application-wide configuration, replacing the traditional `app.module.ts` in a standalone setup.
- `app.routes.ts` : Defines the application's main routes, often using lazy loading for feature modules/pages.
- `/assets` : Stores static assets like images, fonts, or JSON data.
- `/environments` : Contains environment-specific configuration files (e.g., `environment.development.ts`, `environment.production.ts`).

- `index.html` : The main HTML entry file.
- `main.ts` : The primary entry point for the application, bootstrapping it.
- `/node_modules` : Contains all project dependencies installed via npm.
- **Other root files:**
 - `angular.json` : Main project configuration file for the Angular CLI.
 - `package.json` : Lists project dependencies and scripts.
 - `tsconfig.json` : TypeScript compiler configuration.

2. Angular Components & Lifecycle Hooks

Angular components are the main building blocks of an Angular application, responsible for controlling a specific part of the user interface (a view). They are TypeScript classes with associated HTML templates and CSS styles, defined by the `@Component` decorator.

Anatomy of an Angular Component

Each component fundamentally consists of three parts:

- **TypeScript Class:** Contains the component's data and logic, such as properties and methods for handling user input or fetching data.
- **HTML Template:** Defines the component's view and layout using standard HTML combined with Angular-specific template syntax for data binding and control flow.
- **CSS Styles:** Defines the visual appearance of the component's template, scoped by default so styles don't affect other components in the application.
- **Selector:** A CSS selector (e.g., `app-root`) that tells Angular where to insert an instance of the component in the application's HTML.

Key Features of Angular 20 Components

a. Standalone Components:

- No `NgModule` required.
- Components can directly declare their dependencies (other components, directives, pipes).
- Simplified bootstrapping process.

b. Improved CLI Support:

- Angular CLI 20 scaffolds projects with standalone components by default.
- Built-in support for routing, SSR (`--ssr flag`), and SCSS setup.

c. Lifecycle Hooks & Patterns:

- Same familiar hooks (`ngOnInit` , `ngOnDestroy` , etc.), but optimized for standalone usage.
- Cleaner dependency injection and service usage.

Command to generate a component:

```
ng generate component <component-name>
```

Standalone Component Example in Angular 20

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-hello',
  standalone: true,
  imports: [CommonModule],
  template: `<h1>Hello Angular 20!</h1>`,
  styles: [`h1 { color: #1976d2; }`]
})
export class HelloComponent {}
```

Now updated in `app.ts`:

```
@Component({
  selector: 'app-root',
  imports: [RouterOutlet, Hello],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App {
  protected readonly title = signal('app1');
}
```

Now updated in `app.html`:

```
<h1>Om Sai Ram</h1>
<app-hello></app-hello>
```

Component Interaction — Data Binding

Component interaction in Angular relies heavily on data binding. Data binding is a mechanism that synchronizes data between the component's logic (model) and its template (view), ensuring the UI stays updated automatically while reducing the need for manual DOM manipulation.

1. Property Binding

- **Definition:** Passes data from the component class (TypeScript) to the template (HTML).
- **Syntax:** `[property]="expression"`

HTML Template:

```
<img [src]="imageUrl" [alt]="imageDescription">
```

TypeScript Class:

```
export class AppComponent {  
  imageUrl = 'assets/logo.png';  
  imageDescription = 'Company Logo';  
}
```

2. Event Binding

- **Definition:** Captures events from the DOM and sends them to the component class.
- **Syntax:** `(event)="handler"`

HTML Template:

```
<button (click)="onClick()">Click Me</button>
```

TypeScript Class:

```
export class AppComponent {  
  onClick() {  
    console.log('Button clicked!');  
  }  
}
```

3. Two-way Binding

- **Definition:** Combines property binding and event binding, allowing instant synchronization between the component and the template.
- **Syntax:** `[(ngModel)]="property"`

HTML Template:

```
<input [(ngModel)]="username" placeholder="Enter your name">  
<p>Hello, {{ username }}!</p>
```

TypeScript Class:

```
export class AppComponent {
  username = '';
}
```

Component Lifecycle Hooks

Angular components go through a lifecycle from creation to destruction, and Angular provides lifecycle hooks that let you tap into these stages.

Simplified Lifecycle Flow:

```
ngOnChanges → ngOnInit → ngDoCheck → ngAfterContentInit →
ngAfterContentChecked → ngAfterViewInit → ngAfterViewChecked → ngOnDestroy
```

- **ngOnChanges:** Called whenever an `@Input()` property changes. Used to respond to changes in data passed from a parent component.

```
ngOnChanges(changes: SimpleChanges) {
  console.log('Input changed:', changes);
}
```

- **ngOnInit:** Runs once, after the first `ngOnChanges`. Used for initialization logic (fetching data, setting up state).

```
ngOnInit() {
  console.log('Component initialized');
}
```

- **ngDoCheck:** Runs on every change detection cycle. Used for custom change detection beyond Angular's default.

```
ngDoCheck() {
  console.log('Change detection triggered');
}
```

- **ngAfterContentInit:** Called after Angular projects external content into the component (via `<ng-content>`). Used to work with projected content.

```
ngAfterContentInit() {
  console.log('Content projected');
}
```

- **ngAfterContentChecked:** Runs after every check of projected content. Used to respond to changes in projected content.

- **ngAfterViewInit:** Runs after the component's view (and child views) are fully initialized. Used to access child components or DOM elements.

```
ngAfterViewInit() {
  console.log('View initialized');
}
```

- **ngAfterViewChecked:** Runs after every check of the component's view. Used to respond to changes in the view.
- **ngOnDestroy:** Runs just before Angular destroys the component. Used for cleanup tasks (unsubscribing from observables, removing event listeners).

```
ngOnDestroy() {
  console.log('Component destroyed');
}
```

Parent-Child Component Communication (@Input and @Output)

Parent-child communication in Angular is primarily handled using `@Input()` and `@Output()` decorators. These allow data to flow down from parent to child and up from child to parent.

@Input() — Passing Data from Parent to Child: Enables a parent component to bind values to a child component's properties by decorating a property in the child component with `@Input()`.

Child Component (child.component.ts):

```
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-child',
  standalone: true,
  template: `<p>Received: {{ message }}</p>`
})
export class ChildComponent {
  @Input() message: string = '';
}
```

Parent Component (parent.component.html):

```
<app-child [message]="parentMessage"></app-child>
```

Parent Component (parent.component.ts):

```
export class ParentComponent {  
  parentMessage = 'Hello from Parent!';  
}
```

3. Directives and Pipes

Angular 20 uses directives — classes that add behavior to elements — categorized into three main types: Component, Structural, and Attribute. The release emphasizes modern features like the new `@` control flow syntax for structural directives and typed host bindings.

Types of Directives

- **Component Directives:** These are the primary building blocks of Angular applications, each associated with a template and encapsulating a section of the UI. Every `@Component` is a type of directive.
- **Structural Directives:** These change the structure of the DOM layout by adding or removing elements (e.g., `*ngIf`, `*ngFor`, `*ngSwitch`). Angular 20 updates this to a modern JavaScript-like template syntax using `@if`, `@for`, and `@switch`, which is now the preferred, cleaner approach.
- **Attribute Directives:** These modify the appearance or behavior of an existing element, component, or another directive (e.g., `[ngClass]`, `[ngStyle]`, `[(ngModel)]`). Angular 20 introduces support for typed host bindings to enforce type safety and reduce runtime errors.

Key Updates in Angular 20

- **Deprecation Notices:** Angular 20 marks `NgIf`, `NgFor`, `NgSwitch`, `NgSwitchCase`, and `NgSwitchDefault` as deprecated in favor of the new control flow syntax.
- **New Control Flow:** Angular now recommends using the modern `@if`, `@for`, and `@switch` blocks instead of the older `*` prefix syntax.
- **Backward Compatibility:** The old directives still work, but you'll see warnings in the console. They are maintained for migration purposes.

Conditional Rendering Syntax Comparison

Old (Deprecated):

```
<h1 *ngIf="isVisible">Visible Content</h1>
```

New (Recommended):

```
@if (isVisible) {
  <h1>Visible Content</h1>
}
@if (!isVisible) {
  <h1>Invisible Content</h1>
}
```

Component TS File: `isVisible = false;`

Looping Through Collections Syntax Comparison

Old (Deprecated):

```
<ul>
  <li *ngFor="let item of items">{{ item }}</li>
</ul>
```

New (Recommended):

```
<ul>
  @for (item of items; track item) {
    <li>{{ item }}</li>
  }
</ul>
```

Attribute Directives: ngClass, ngStyle, ngModel

ngClass: Dynamically adds or removes CSS classes based on component state expression.

```
<!-- Apply a single class -->
<div [ngClass]=" 'highlight' ">Highlighted Text</div>

<!-- Apply multiple classes conditionally -->
<div [ngClass]="{ 'active': isActive, 'disabled': isDisabled }">
  Conditional Classes
</div>
```

ngStyle: Dynamically sets inline styles depending on component properties.

```
<p [ngStyle]="{ 'color': textColor, 'font-size.px': fontSize }">Styled with ngStyle!</p>
<button (click)="toggleColor()">Toggle Color</button>
```

Implementation Example:

```

@Component({
  selector: 'app-contact',
  imports: [CommonModule],
  templateUrl: './contact.html',
  styleUrls: ['./contact.css'],
})
export class Contact {
  textColor = 'blue';
  fontSize = 18;

  toggleColor() {
    this.textColor = this.textColor === 'blue' ? 'red' : 'blue';
  }
}

```

ngModel: Enables two-way data binding between form inputs and component properties. Best for simple cases, though Angular encourages Reactive Forms for complex form handling.

```

<input [(ngModel)]="username" placeholder="Enter your name">
<p>Hello, {{ username }}!</p>

```

TypeScript Class: `username: string = '';`

Custom Directives

With Angular 20, directives are standalone by default, meaning you don't need to declare them inside an NgModule; you just import them directly into the component where you want to use them.

Generate a directive using Angular CLI:

```

ng generate directive highlight

```

Custom Attribute Directive Logic (highlight.directive.ts):

```
import { Directive, ElementRef, Renderer2, HostListener } from '@angular/core';

@Directive({
  selector: '[appHighlight]',
  standalone: true
})
export class HighlightDirective {
  constructor(private el: ElementRef, private renderer: Renderer2) {}

  @HostListener('mouseenter') onMouseEnter() {
    this.renderer.setStyle(this.el.nativeElement, 'background-color', 'yellow');
  }

  @HostListener('mouseleave') onMouseLeave() {
    this.renderer.removeStyle(this.el.nativeElement, 'background-color');
  }
}
```

- `@Directive` marks it as a directive.
- `ElementRef` gives access to the DOM element.
- `Renderer2` safely manipulates styles.
- `@HostListener` listens to events on the host element.

Usage: `<p appHighlight>Hover over me!</p>` (Ensure `imports: [HighlightDirective]` in TS).

Custom Structural Directive: Structural directives change DOM structure using `TemplateRef` and `ViewContainerRef`.

```
import { Directive, Input, TemplateRef, ViewContainerRef } from '@angular/core';

@Directive({
  selector: '[appUnless]',
  standalone: true
})
export class UnlessDirective {
  @Input() set appUnless(condition: boolean) {
    if (!condition) {
      this.vcRef.createEmbeddedView(this.templateRef);
    } else {
      this.vcRef.clear();
    }
  }

  constructor(private templateRef: TemplateRef<any>, private vcRef: ViewContainerRef) {}
}
```

Usage: `<p *appUnless="isVisible">This shows only when isVisible is false</p>`

Pipes

In Angular, pipes are functions used in templates to transform and format data before it is displayed to the user without modifying the underlying data in the component class. (Requires `imports: [CommonModule]` for built-in pipes in standalone components).

Built-in Pipes:

- **Date Pipe:** Formats dates according to locale rules.

```
<p>{{ today | date }}</p>
<p>{{ today | date:'fullDate' }}</p>
<p>{{ today | date:'shortTime' }}</p>
```

- **Uppercase Pipe:** Transforms text to uppercase.

```
<p>{{ 'angular pipes' | uppercase }}</p>
```

- **Currency Pipe:** Formats numbers as currency values.

```
<p>{{ 1234.56 | currency }}</p>
<p>{{ 1234.56 | currency:'EUR' }}</p>
<p>{{ 1234.56 | currency:'INR':'symbol':'1.2-2' }}</p>
```

Creating Custom Pipes: A pipe is a TypeScript class with a `@Pipe` decorator implementing a `transform` function containing the logic.

Command: `ng g pipe bywords`

```

import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'bywords',
  standalone: true
})
export class BywordsPipe implements PipeTransform {
  transform(value: number, opttype: string): string {
    let result = '';
    while(value > 0) {
      const digit = value % 10;
      switch(digit) {
        case 0: result = 'zero ' + result; break;
        case 1: result = 'one ' + result; break;
        case 2: result = 'two ' + result; break;
        case 3: result = 'three ' + result; break;
        case 4: result = 'four ' + result; break;
        case 5: result = 'five ' + result; break;
        case 6: result = 'six ' + result; break;
        case 7: result = 'seven ' + result; break;
        case 8: result = 'eight ' + result; break;
        case 9: result = 'nine ' + result; break;
      }
      value = Math.floor(value / 10);
    }
    if(opttype === 'upper') {
      result = result.toUpperCase();
    }
    return result;
  }
}

```

Usage: `<h1>Value of 12345 is {{ 12345 | bywords:'upper' }}</h1>` (Import `BywordsPipe` in component config).

Other Custom Pipe Examples:

```

// Titlecase Pipe Example
@Pipe({ name: 'titlecase' })
export class TitlecasePipe implements PipeTransform {
  transform(value: string): string {
    let words = value.split(' ');
    let title = '';
    for(let i=0; i<words.length; i++) {
      title = title + words[i].charAt(0).toUpperCase() +
words[i].substring(1).toLowerCase() + ' ';
    }
    return title.trim();
  }
}

// Star Pipe Example
@Pipe({ name: 'star' })
export class StarPipe implements PipeTransform {
  transform(value: string): string {
    return `★ ${value} ★`;
  }
}

// Reverse Pipe Example
@Pipe({ name: 'reverse' })
export class ReversePipe implements PipeTransform {
  transform(value: string): string {
    let reverse = '';
    for (let i = value.length - 1; i >= 0; i--) {
      reverse += value[i];
    }
    return reverse;
  }
}

```

Registering Custom Pipe (Traditional Module Setup):

```

import { UppercasePipe } from './custom/uppercase.pipe';

@NgModule({
  declarations: [AppComponent, UppercasePipe],
  imports: [BrowserModule],
  bootstrap: [AppComponent]
})
export class AppModule {}

```

Custom Pipe with Parameters Example:

```
@Pipe({ name: 'appendText' })
export class AppendTextPipe implements PipeTransform {
  transform(value: string, append: string): string {
    return `${value} ${append}`;
  }
}
```

Usage: `<p>{{ 'Angular' | appendText: 'v20' }}</p>`

4. Template-driven Forms & Form Validation

What Are Template-driven Forms?

- They rely on Angular directives (`ngModel` , `ngForm`) inside the HTML template.
- Best suited for simple forms with fewer validations.
- Form logic is mostly handled in the HTML template, not in the component class.

Import Required Module: In traditional apps, add `FormsModule` in `app.module.ts` :

```
import { FormsModule } from '@angular/forms';

@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule, FormsModule, CommonModule],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

Creating a Template-driven Form

Template (app.component.html):

```

<h2>Template Driven Form</h2>
<form #userForm="ngForm" (ngSubmit)="onSubmit(userForm)">
  <div>
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" ngModel required minlength="3"
#name="ngModel">
    <div *ngIf="userForm.controls['name']?.invalid && (userForm.controls['name']?.touched
|| userForm.controls['name']?.dirty)">
      Name is required (min 3 chars).
    </div>
  </div>

  <div>
    <label for="email">Email:</label>
    <input type="email" id="email" name="email" ngModel required email #email="ngModel">
    <div *ngIf="email.invalid && (email.touched || email.dirty)">
      Valid email is required.
    </div>
  </div>

  <button type="submit" [disabled]="userForm.invalid">Submit</button>
</form>

<p>Form Value: {{ userForm.value | json }}</p>

```

Component TypeScript (app.component.ts):

```

import { Component } from '@angular/core';
import { NgForm } from '@angular/forms';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html'
})
export class AppComponent {
  onSubmit(form: NgForm) {
    console.log('Form Submitted!', form.value);
    alert('Form Submitted Successfully!');
  }
}

```

Key Features and Form State Tracking

- **Two-way binding with `[(ngModel)]`:** `<input type="text" [(ngModel)]="username" name="username">`
- **Validation directives:** `required`, `minlength`, `maxlength`, `pattern`.

- **Form state tracking properties:** `userForm.valid`, `userForm.invalid`, `userForm.touched`, `userForm.value`.

Comprehensive Form Validation Demo (with Custom Validation Functions)

```
<form #userForm="ngForm" (ngSubmit)="onSubmit(userForm)">
  <div>
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" ngModel required minlength="3"
#name="ngModel">
    <div *ngIf="name.invalid && name.touched">
      <small *ngIf="name.errors?.['required']">Name is required.</small>
      <small *ngIf="name.errors?.['minlength']">Minimum 3 characters required.</small>
    </div>
  </div>

  <div>
    <label for="email">Email:</label>
    <input type="email" id="email" name="email" ngModel required email #email="ngModel">
    <div *ngIf="email.invalid && email.touched">
      <small *ngIf="email.errors?.['required']">Email is required.</small>
      <small *ngIf="email.errors?.['email']">Enter a valid email.</small>
    </div>
  </div>

  <div>
    <label for="age">Age:</label>
    <input type="number" id="age" name="age" ngModel required #age="ngModel">
    <div *ngIf="age.touched && !isValidAge(age.value)">
      <small>Age must be between 18 and 60.</small>
    </div>
  </div>

  <button type="submit" [disabled]="userForm.invalid">Submit</button>
</form>
```

TypeScript class custom validation helper:

```
export class AppComponent {
  onSubmit(form: NgForm) {
    console.log('Form Submitted!', form.value);
    alert('Form Submitted Successfully!');
  }

  isValidAge(age: number): boolean {
    return age >= 18 && age <= 60;
  }
}
```

5. Dependency Injection and Services

Dependency Injection (DI) is a design pattern where a class receives its dependencies from an external source rather than creating them itself. In Angular, DI is built straight into the framework and is used to provide services across components and directives, promoting loose coupling, testability, reusability, and scalability.

A **Service** in Angular is typically a TypeScript class with a focused purpose, such as fetching data, handling business logic, or managing application state. Services are decorated with `@Injectable()` to mark them as part of the DI system and to allow Angular's optimization (tree-shaking).

Key DI Concepts

- **Separation of Concerns:** Components focus solely on UI logic, while services handle data and business logic.
- **Reusability:** A single service instance can be used across multiple components, ensuring data consistency and reducing code duplication.
- **Injector:** An abstraction that stores and resolves instances of dependencies. Angular creates a root injector at application bootstrap and additional child injectors as needed, forming a hierarchical tree.
- **Provider:** An object that tells the injector how to create a dependency's instance. The most common provider is the service class itself.
- **Injection Token:** A unique identifier used by the injector to link a request for a dependency with its corresponding provider.

Creation Command: `ng generate service service-name`

Handling Async Observable Data Sets in Services

An Observable in Angular (via RxJS) represents a stream of values that can be observed over time. Unlike a Promise, which resolves once, an Observable can emit multiple values asynchronously and allows you to react to changes, events, or data updates. It is a lazy push-based collection that pushes data to its subscribers.

LoggerService Example:

```
import { Injectable } from '@angular/core';
import { Observable, of } from 'rxjs';

@Injectable({
  providedIn: 'root' // makes it available app-wide as a singleton
})
export class LoggerService {
  calculate = (a: number, b: number): number => a + b; // returns normal data

  display = (): Observable<string> => {
    return of("Hello world"); // returns Observable data stream
  }
}
```

Component Consumption Example:

```
import { Component } from '@angular/core';
import { UserService } from '../userservice';

@Component({
  selector: 'app-login',
  templateUrl: './login.html',
  styleUrls: ['./login.css'],
  providers: [UserService] // Component level scoped instance provider
})
export class Login {
  constructor(private userService: UserService) {}

  fun1 = () => {
    let result: number = this.userService.calculate(12, 23);
    console.log(result);
  }

  fun2 = () => {
    this.userService.display().subscribe({
      next: (data) => { console.log(data); },
      error: (error) => { console.log(error); }
    });
  }
}
```

6. Angular HTTP Client & REST API Integration

`HttpClient` is a built-in Angular service (`@angular/common/http`) for communicating with backend services over HTTP. It allows your app to send requests and handle responses from REST APIs or servers.

Key Features:

- Strongly typed response handling (e.g., `Observable<Post[]>`)
- Simplified error handling capabilities
- Request/response interceptors (for logging, authentication header attachment, etc.)

Setting Up HttpClientModule

`HttpClientModule` is an Angular module that registers `HttpClient` and its related providers to make it available for dependency injection throughout your app.

In AppModule (Traditional Architecture Setup):

```
import { BrowserModule } from '@angular/platform-browser';
import { HttpClientModule } from '@angular/common/http';

@NgModule({
  imports: [BrowserModule, HttpClientModule],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

In Standalone Components Configuration:

```
@Component({
  standalone: true,
  imports: [HttpClientModule],
  templateUrl: './app.component.html'
})
export class AppComponent {}
```

REST API Endpoints Mock Environment Setup (JSON Server)

```
npm install -g json-server
```

Create a file named `db.json` under the project root folder:

```
{
  "posts": [
    { "id": 1, "title": "First Post", "body": "Hello Angular", "userId": 1 },
    { "id": 2, "title": "Second Post", "body": "Learning HttpClient", "userId": 2 }
  ]
}
```

Run the server instance: `json-server --watch db.json --port 3000`

Active API Endpoints Available:

- `GET http://localhost:3000/posts`
- `POST http://localhost:3000/posts`
- `PUT http://localhost:3000/posts/:id`
- `DELETE http://localhost:3000/posts/:id`

Creating the Angular Service for full CRUD Operations

Command: `ng g s apiservice`

Implementation Code (apiservice.service.ts):

```

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root',
})
export class Apiservice {
  private baseUrl = 'http://localhost:3000';

  constructor(private http: HttpClient) {}

  getPosts(): Observable<any> {
    return this.http.get(`${this.baseUrl}/posts`);
  }

  createPost(postData: any): Observable<any> {
    return this.http.post(`${this.baseUrl}/posts`, postData);
  }

  updatePost(id: number, postData: any): Observable<any> {
    return this.http.put(`${this.baseUrl}/posts/${id}`, postData);
  }

  deletePost(id: number): Observable<any> {
    return this.http.delete(`${this.baseUrl}/posts/${id}`);
  }
}

```

Angular Component and Form Setup for Data Operations

Component Implementation Class (client.component.ts):

```

import { ChangeDetectorRef, Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { ApiService } from '../apiservice';

@Component({
  selector: 'app-client',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './client.html',
  styleUrls: ['./client.css'],
})
export class Client implements OnInit {
  posts: any[] = [];
  selectedPost: any = { title: '', body: '', userId: 1 };
  newPost: any = { title: '', body: '', userId: 1 };
  isEditing = false;

  constructor(private apiService: ApiService, private cdr: ChangeDetectorRef) {}

  ngOnInit() {
    this.loadPost();
  }

  loadPost() {
    this.apiService.getPosts().subscribe(data => {
      this.posts = data;
      console.log(this.posts);
      this.cdr.markForCheck();
    });
  }

  addPost() {
    this.apiService.createPost(this.newPost).subscribe({
      next: (created) => {
        console.log('Created:', created);
        this.posts.push(created);
        this.newPost = { title: '', body: '', userId: 1 };
        this.cdr.markForCheck();
      },
      error: err => console.error('Create failed', err)
    });
  }

  editPost(post: any) {
    this.selectedPost = { ...post };
    this.isEditing = true;
  }
}

```

```

savePost() {
  if (this.isEditing) {
    this.apiService.updatePost(this.selectedPost.id, this.selectedPost).subscribe({
      next: updated => {
        const index = this.posts.findIndex(p => p.id === updated.id);
        if (index > -1) this.posts[index] = updated;
        this.resetForm();
        this.cdr.markForCheck();
      },
      error: err => console.error('Update failed', err)
    });
  } else {
    this.apiService.createPost(this.selectedPost).subscribe({
      next: created => {
        this.posts.push(created);
        this.resetForm();
        this.cdr.markForCheck();
      },
      error: err => console.error('Create failed', err)
    });
  }
}

deletePost(id: any) {
  alert('Delete clicked for id=' + id);
  this.apiService.deletePost(id).subscribe({
    next: () => {
      console.log('Deleted:', id);
      this.posts = this.posts.filter(p => p.id !== id);
      this.cdr.markForCheck();
    },
    error: err => console.error('Delete failed', err)
  });
}

resetForm() {
  this.selectedPost = { title: '', body: '', userId: 1 };
  this.isEditing = false;
}
}

```

Template Markup (client.html):

```

<!-- Display list -->
<ul>
  @for (post of posts; track post.id) {
    <li>
      {{ post.id }} || {{ post.title }} || {{ post.body }} || By: {{ post.userId }}
      <button (click)="editPost(post)">Edit</button>
      <button (click)="deletePost(post.id)">Delete</button>
    </li>
  }
</ul>

<!-- Add Form -->
<h2>Add New Post</h2>
<form (ngSubmit)="addPost()" #postForm="ngForm">
  <label>Title:</label>
  <input type="text" [(ngModel)]="newPost.title" name="title" required />
  <label>Body:</label>
  <textarea [(ngModel)]="newPost.body" name="body" required></textarea>
  <button type="submit" [disabled]="postForm.invalid">Add Post</button>
</form>

```

Role-Based JWT Authentication Integration Changes

If the endpoint demands security protocols using JWT token architectures, add these dynamic helper configurations inside the service layer:

```

import { HttpHeaders } from '@angular/common/http';

private getAuthHeaders(): HttpHeaders {
  const token = localStorage.getItem('token'); // Or sessionStorage.getItem('token')
  return new HttpHeaders({
    'Authorization': `Bearer ${token}`
  });
}

getPosts(): Observable<any> {
  return this.http.get(`${this.baseUrl}/posts`, { headers: this.getAuthHeaders() });
}

createPost(postData: any): Observable<any> {
  return this.http.post(`${this.baseUrl}/posts`, postData, { headers:
this.getAuthHeaders() });
}

```

Handling API Responses: Observables vs. Promises

In Angular 20, API responses can be handled using both Promises and Observables. However, **Observables are the preferred approach** because Angular's HttpClient returns Observables by default, offering much richer options (streaming, cancellation, rich operators) compared to Promises, which resolve only once.

- **Promise Characteristics:**

- Execution: Executes immediately when created.
- Values: Resolves to a single value only.
- Cancellation: Non-cancellable structure.
- Operators: Extremely limited chaining capabilities (`.then` , `.catch`).

- **Observable Characteristics:**

- Execution: Lazy; executes only when explicitly subscribed to.
- Values: Can emit multiple values over time streams.
- Cancellation: Highly cancellable via `unsubscribe()` invocations to stop execution loops.
- Operators: Extremely rich structural RxJS set (`map` , `filter` , `switchMap` , etc.).

Code Implementations Comparison:

Using Promises Architecture:

```
this.http.get('https://api.example.com/data')
  .toPromise()
  .then(response => { console.log('Data:', response); })
  .catch(error => { console.error('Error:', error); });
```

Using Observables Standard Implementation:

```
this.http.get('https://api.example.com/data')
  .subscribe({
    next: (response) => console.log('Data:', response),
    error: (error) => console.error('Error:', error),
    complete: () => console.log('Request complete')
  });
```

Error Handling and Retry Strategies

Combine `HttpClient` with RxJS operators like `catchError` , `retry` , and `retryWhen` to gracefully handle REST API errors. Best practice rules state that you should centralize error handling within interceptors, give friendly feedback visual components to users, and apply retry loops strictly for transient issues like network drops.

1. Basic Error Handling with `catchError`: Intercept errors at the individual service method layout point.

```
import { HttpResponse } from '@angular/common/http';
import { throwError } from 'rxjs';
import { catchError } from 'rxjs/operators';

this.http.get('/api/data').pipe(
  catchError((error: HttpResponse) => {
    console.error('Error occurred:', error);
    return throwError(() => new Error('Something went wrong, please try again.'));
  })
);
```

2. Retry Strategies: Use `retry` for simple counter attempts, or `retryWhen` for custom delay parameters.

```
// Simple retry loop example
this.http.get('/api/data').pipe(
  retry(3), // retries 3 times before failing forward
  catchError(this.handleError)
);

// Advanced retryWhen conditional strategy example
import { delay, take, concatMap } from 'rxjs/operators';
import { of } from 'rxjs';

this.http.get('/api/data').pipe(
  retryWhen(errors =>
    errors.pipe(
      delay(2000), // wait 2 seconds before each retry attempt
      take(3),     // retry maximum up to 3 times total
      concatMap((error, count) => count === 3 ? throwError(() => error) : of(error))
    )
  )
);
```

3. Comprehensive Service with Full Error Handling & Retry Code:

```

import { Injectable } from '@angular/core';
import { HttpClient, HttpResponse } from '@angular/common/http';
import { catchError, retryWhen, delay, take, throwError } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class ApiService {
  private apiUrl = 'https://example.com/api/data';

  constructor(private http: HttpClient) {}

  getData() {
    return this.http.get(this.apiUrl).pipe(
      retryWhen(errors =>
        errors.pipe(
          delay(2000),
          take(3)
        )
      ),
      catchError(this.handleError)
    );
  }

  private handleError(error: HttpResponse) {
    if (error.error instanceof ErrorEvent) {
      console.error('Client-side error:', error.error.message);
    } else {
      console.error(`Server returned code ${error.status}, body was:`, error.error);
    }
    return throwError(() => new Error('Something went wrong; please try again later.'));
  }
}

```

Centralized Global Error Handling via HttpInterceptors

```
import { Injectable } from '@angular/core';
import { HttpInterceptor, HttpRequest, HttpHandler, HttpEvent, HttpResponse } from '@angular/common/http';
import { Observable, throwError } from 'rxjs';
import { catchError } from 'rxjs/operators';

@Injectable()
export class ErrorInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
    return next.handle(req).pipe(
      catchError((error: HttpResponse) => {
        let errorMsg = '';
        if (error.status === 0) {
          errorMsg = 'Network error: Please check your connection.';
        } else if (error.status === 404) {
          errorMsg = 'Resource not found.';
        } else if (error.status === 500) {
          errorMsg = 'Server error. Please try again later.';
        } else if (error.status === 401) {
          errorMsg = 'Unauthorized access.';
        } else {
          errorMsg = `Unexpected error: ${error.message}`;
        }
        console.error(errorMsg);
        return throwError(() => new Error(errorMsg));
      })
    );
  }
}
```

Register the interceptor inside your `AppModule` provider configurations setup block:

```
import { HTTP_INTERCEPTORS } from '@angular/common/http';
import { ErrorInterceptor } from './error.interceptor';

@NgModule({
  providers: [
    { provide: HTTP_INTERCEPTORS, useClass: ErrorInterceptor, multi: true }
  ]
})
export class AppModule {}
```

Interceptor Component Usage Consumption Pattern:

```

import { Component, OnInit } from '@angular/core';
import { ApiService } from '../api.service';

@Component({
  selector: 'app-root',
  template: `
    <div *ngIf="errorMessage">{{ errorMessage }}</div>
    <div *ngIf="data">{{ data | json }}</div>
  `
})
export class AppComponent implements OnInit {
  data: any;
  errorMessage: string | null = null;

  constructor(private apiService: ApiService) {}

  ngOnInit() {
    this.apiService.getData().subscribe({
      next: (res) => this.data = res,
      error: (err) => this.errorMessage = err.message
    });
  }
}

```